

When Does Repartitioning Pay? Measurement Pitfalls in Small CPU Language-Model Serving (Working Draft)

Anonymous Authors

Abstract—This working draft retains incomplete local experiments explicitly. Splitting a language model that fits on one machine creates a deployment choice rather than a capacity requirement. We study that choice with Shardwise, a CPU runtime for contiguous-layer inference, voluntary repartitioning, and replay recovery. An audit of 65 two-VM Qwen3-0.6B runs finds that objective choice changes throughput substantially, but an apparent 25% fault-phase adaptation gain shrinks to about 5–7% over the complete run; adaptation under injected link delay loses about 6–8%. The payback gate does not demonstrate an advantage over hysteresis. We distinguish ideal service capacity from finite-session completion, expose a three-worker counterexample to a simple concurrency heuristic, and implement an exact envelope planner. New local experiments compare the pipeline with complete-model replicas and unsplit CPU serving under an explicit resource budget. Position-controlled recovery checks retain exact greedy outputs while measuring token stalls. Worker queue timing separates a real-model application residual into wait and remaining overhead. These results support an empirical account of baseline choice, transition accounting, and controller limitations, rather than a general claim that partitioning outperforms replication.

Index Terms—LLM inference, pipeline parallelism, CPU serving, performance measurement, recovery

I. INTRODUCTION

A small language model can fit on each of several available machines. In this setting, splitting layers is one serving option; complete-model replication is another. Independent requests can overlap across pipeline stages, while a single autoregressive request traverses every active stage for each token. Moving layer boundaries introduces another tradeoff: a better steady-state placement may arrive too late to repay its reconfiguration cost.

Shardwise provides a compact implementation in which these costs can be inspected. A Go controller places contiguous decoder-layer ranges; Python workers execute them over gRPC; a router tracks independent request histories and rebuilds their KV caches after a generation change. Its techniques are established. The question here is how measurement and baseline choices affect the conclusions drawn from a small running service.

The original two-VM record contains 65 runs and 939 requests. We audited that record without changing the raw data. Its most useful findings are negative: whole-run adaptation gains are smaller than fault-phase gains; moving under delay hurts; the transition-cost gate does not outperform a simpler policy in the observed medians; and a remaining-work horizon

does not describe a service whose clients keep admitting new requests. One-thread whole-model execution also leaves a central baseline question unresolved: if the model fits on both machines, why not replicate it?

We address that question with a separately reported local baseline matrix, and improve the implementation’s diagnostic and correctness support. The contributions are (i) an audited empirical study with explicit claim boundaries, (ii) a resource-accounted real-model replica comparison, (iii) position-controlled replay checks and queue-aware measurements, and (iv) a corrected planner for the ideal finite-concurrency envelope. Exact optimization of this model does not imply optimal placement on the real runtime. We do not claim a new general migration, recovery, or partitioning principle.

II. RELATED WORK

Pipeline parallelism and contiguous placement precede this system [1–3]. EdgeShard separates sequential latency from pipelined throughput and uses dynamic programming with device and memory constraints [4]. Petals demonstrates distributed block serving and recovery on consumer devices [5]. Those systems address broader capacity aggregation than the small model studied here.

Migration payback is established. DiSCo admits generation handover using remaining decode savings and migration overhead; STAR filters migrations that cannot amortize their cost [6, 7]. ServerlessLLM reconstructs state from token history [8]. SpotServe combines workload-aware serving, migration, reparallelization and preemption recovery [9]. These capabilities prevent presenting their combination as an independent novelty claim.

DynoPipe jointly considers workload, queueing, compute, network conditions and pipeline switching in a heterogeneous edge/cloud GPU system [10]. Its RTX 3090 edge and A40 cloud deployment differs from our CPU testbed. FlexPipe adapts pipeline granularity to request patterns and maintains cache consistency during refactoring [11]. Shardwise’s distinction is narrower: an inspectable CPU implementation and evidence about short-run disruption, baseline dispatch, and misleading telemetry. No matched experiment with these prior systems was performed.

An August 2026 preprint examines pre-compiled OpenVINO pipeline shards and multiuser interleaving on Intel AI PCs [12]. It provides additional commodity-device context; we do not

treat it as verified peer-reviewed acceptance or a matched comparison with our runtime.

Performance methodology calls for resource-matched alternatives, variability and a clear analysis unit [13, 14]. We use runs as units, retain failures, and report phase and whole-run metrics separately. Requests within a run do not increase the independent sample size.

III. RUNTIME AND DECISION MODEL

A. Execution and recovery

Workers hold half-open ranges of decoder layers. The first active stage embeds tokens; the last applies normalization and the output head. Each worker serializes compute with one lock. Independent request threads can overlap between stages, but there is no continuous batching or speculative decoding. The router passes activations through an ordered chain and receives the greedy next token. In cached mode, later steps send one position per request.

Every assignment has a generation. Workers reject a forward from another generation. On a relayout, missing cache, or unreachable worker, the router obtains the available layout and replays the entire accumulated token history under a fresh request identifier. Already accepted output tokens remain in the history. This mechanism requires enough surviving memory to hold the assigned model; it does not protect the router or controller machine.

Node agents report compute-speed and network observations to the controller. Kernel TCP telemetry and application timing are separate sources. The latter historically used RPC time minus worker compute time, which also prices time waiting for the worker. The revision exposes compute-lock wait separately and offers an opt-in queue-corrected residual. That remaining residual includes serialization, dispatch and transport; it is not pure network RTT.

B. Placement and the concurrency qualification

For a nonempty worker range $[a, b)$, the cost model prices layer compute, endpoint modules and the link into that worker:

$$d_i(a, b) = \frac{(b - a)c(C) + e\mathbf{1}_{a=0} + h\mathbf{1}_{b=L}}{s_i} + \ell_i. \quad (1)$$

The local worker has zero link term. Empty workers are skipped and cost zero. The model assumes fixed worker order, additive stage costs, and no explicit memory constraint. Measured context costs interpolate $c(C)$. Sum placement minimizes $P = \sum_i d_i$; saturated throughput placement minimizes $B = \max_i d_i$. The ordered DP combines a prefix optimum with each possible last range, using addition for P and maximum for B , in $O(WL^2)$ time.

For Q non-speculative requests, ideal aggregate throughput obeys

$$X_Q \leq \min(1/B, Q/P), \quad j_Q = \max(B, P/Q). \quad (2)$$

Each token needs service at every stage, giving the bottleneck bound; the closed network has at least P cycle time, giving the

request-count bound through Little's law [15, 16]. These are capacity bounds, not achievable-throughput predictions. Prefill, queues, interpreter overhead and changing request lengths remain outside the envelope.

The historical automatic policy selected sum at $Q \leq 1$ and bottleneck otherwise. This two-worker heuristic does not generalize. Nine unit-cost layers on three equal workers with unit costs into remote nonempty workers give a minimum bottleneck $B = 4$ with $P = 11$. At $Q = 2$, its envelope cost is 5.5. Two active stages can instead give $B = 5, P = 10$, for cost 5.

The revised automatic planner minimizes j_Q directly. It enumerates possible stage-cost caps b and runs a sum DP restricted to stages no greater than b . The least achievable sum $P(b)$ decreases as the cap increases. Before the first crossing $b \geq P(b)/Q$, the score is $P(b)/Q$; after it, the score is b . Comparing the crossing and its predecessor yields the exact optimum. Binary search requires $O(\log(WL^2))$ constrained DPs, each $O(WL^2)$, after sorting the possible caps. Exhaustive enumeration of 1,000 small random instances checks heterogeneity, empty stages, endpoints and concurrency. The historical results below use the original planner.

C. Payback and finite cohorts

Voluntary moves first pass an improvement threshold and cooldown. The historical gate estimates transition time as

$$T = F + CpL, \quad (3)$$

with fixed load/orchestration cost F , configured context C , and replay cost p per token-layer. It prices the entire chain, including unmoved layers. However, the configured C does not sum simultaneous request histories. The revision offers a live-context forecast $F + pL \sum_r C_r$, where each C_r contains prompt and accepted output tokens. This estimates serial replay work; overlap, weights, serialization and orchestration still require calibration. Unavailable live-context telemetry holds voluntary moves but permits recovery.

For positive current and candidate token costs $j_c > j_n$, horizon H and relative capacity margin $m \geq 0$, the historical fluid gate accepts when

$$\frac{(H - T)_+}{j_n} > (1 + m) \frac{H}{j_c}. \quad (4)$$

Multiplying positive denominators shows that, for $T > 0$, acceptance is equivalent to $\Delta_m = j_c - (1 + m)j_n > 0$ and $H > Tj_c/\Delta_m$. Capping the horizon at $\min(H, Rj_c)$ additionally requires $R > T/\Delta_m$. This is an algebraic statement about the chosen capacity surrogate. It is not a necessary-and-sufficient guarantee for real session completion. A finite cohort cannot produce more than its R remaining tokens; concurrency also decreases as requests finish. For constant costs, the zero-margin makespan comparison is $T + Rj_n < Rj_c$. A completion-time margin would require a separately stated condition, such as $(1 + m)(T + Rj_n) < Rj_c$.

IV. EXPERIMENTAL METHOD

A. Historical two-VM evidence

The audited record uses Qwen3-0.6B, 28 layers, FP32 CPU execution and KV caching on two Azure VMs with two vCPUs and about 8 GiB each. One worker thread per machine is used. The coordinator hosts the controller, router and one worker. The matrices comprise 20 objective runs, 12 demand runs, 30 adaptation runs and three single-device runs. The total is 65 runs, 939 requests and 19 recorded request failures. Distinct source snapshots are retained and matrices are not pooled. Raw host labels differ from the manuscript labels and are documented in the artifact.

Objective runs compare sum and bottleneck placement at concurrency one and two, with stable links or 120 ms injected delay. Adaptation compares static, hysteresis, gate and forced-gate arms under CPU quota, link delay or worker loss. The demand matrix compares fixed throughput placement and historical automatic placement with fixed or remaining-work horizons. These comparisons have only two or three independent repetitions per condition.

Whole-run throughput divides successful output tokens by elapsed time through the last request’s completion, including drain and failures. Historical fault-phase throughput assigns tokens by request completion within the logged fault window, not by streamed token timestamps. This convention can move work across phase boundaries. Median request latency and router TTFT are supporting metrics; TTFT does not measure client streaming delivery.

B. New local evidence

The new baseline matrix is an additional evidence stratum. All arms use the same cached FP32 model, prompts, greedy output, worker backend and HTTP/gRPC path. The arms are one-thread unsplit serving, two-thread unsplit serving, two complete-model one-thread replicas, and two one-thread pipeline workers with fixed 18/10 placement. Inference is restricted to two host CPUs; the one-thread unsplit arm leaves one unused. The other two host CPUs provide coordination consistently. Replicas reserve the first available endpoint for an entire request, including dispatch wait in client latency.

Ten restart blocks randomize arm order. Contexts are 32 and 128 tokens; output length is 16; concurrency is one, two, four or eight. Each run serves at least four requests and at least as many as concurrency. The program saves prompts, reference outputs, model revision, package versions, source hashes, CPU affinities and process memory. Every returned sequence is checked against unsplit Hugging Face greedy output. Block bootstrap intervals resample paired run contrasts, not individual requests. They are descriptive, unadjusted intervals; the study has no prespecified power calculation or equivalence test.

The fixed inputs are seeded pseudorandom token IDs, rather than a natural-language task suite. This controls sequence length and permits exact reference checks; it does not establish answer quality or coverage of application workloads. The local baseline campaigns were stopped before completion at the

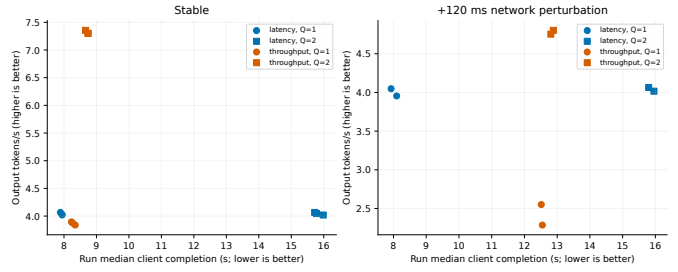


Fig. 1. Historical objective runs on two VMs. Each point is a run. Whole-run throughput and completion include drain. Sum and bottleneck arms activate different amounts of compute; this is not a replica comparison.

user’s request. The EPYC 7763 campaign retains 129 of 320 planned runs; a separate EPYC 9V74 campaign retains 37. These strata are not pooled, and no completed ten-block local result is claimed. Workspace restarts and changing hardware required checkpoint resumes; physical host identity across restarts was not independently authenticated.

The pilot is kept separate because builds and cache cleanup ran concurrently. Local loops do not reproduce two physical machines or Azure link behavior. The complete-model baselines are useful matched alternatives within the runtime, but an optimized external engine remains unevaluated.

Recovery checks manually inject relayout, remote-worker loss, or remote-worker restart at accepted-token positions 1, 4 and 12, with one and two active requests. Actual worker processes and gRPC are used. The surviving worker loads the full model after loss. These checks validate replay sequencing and internal stalls; they do not measure heartbeat detection, controller response, router loss or delivery through a streaming HTTP API.

V. RESULTS

A. Objective choice helps an internal baseline

With two stable concurrent requests, historical bottleneck placement achieves 7.303 versus 4.046 tokens/s for sum placement, an 80.5% gain. With one request, they achieve 3.875 and 4.029 tokens/s. Under injected delay at concurrency one, sum placement achieves 4.001 versus 2.419 tokens/s: 65.4% higher throughput, or a 39.5% loss in the reverse comparison. Median completion time falls from 12.531 to 8.012 s, a 36.1% reduction.

These results show that objective choice matters inside this runtime. They also compare different amounts of active compute: sum placement can put the whole model on one worker, while bottleneck placement activates both. They therefore do not establish superiority over resource-matched serving.

B. The gate has no demonstrated benefit over hysteresis

Under CPU slowdown, hysteresis achieves 5.269 versus 4.215 fault-phase tokens/s for static placement, a 25.0% gain. Whole-run throughput changes from 5.744 to 6.133, only 6.8%. The gate achieves 6.041 whole-run tokens/s. Under delay, static

TABLE I
HISTORICAL WHOLE-RUN THROUGHPUT MEDIANS (TOKENS/S).
INDEPENDENT REPETITIONS PER ARM: THREE FOR CPU AND DELAY; TWO
FOR LOSS. THE DIFFERENCES DO NOT ESTABLISH STATISTICAL
SIGNIFICANCE.

Condition	Static	Hysteresis	Gate
CPU slowdown	5.744	6.133	6.041
Link delay	6.554	6.131	6.055
Worker loss	3.271	5.443	5.409

achieves 6.554, hysteresis 6.131 and gate 6.055 tokens/s. Both adaptive arms lose about 6–8% over the complete run despite similar fault-phase throughput. Under worker loss, gate and hysteresis restore service with no failed requests; static records 19. The fault-blind static comparison does not establish better availability than replication or another recovery policy.

No observed median demonstrates a gate advantage. With only two or three repetitions, the small differences between gate and hysteresis are not evidence of a statistically established loss either. The defensible finding is that adding a reasonable cost gate did not demonstrate better serving in this record.

At concurrency two in the demand matrix, fixed throughput placement achieves 7.830 tokens/s, historical automatic fixed-horizon placement 5.944, and automatic remaining-work placement 5.729. Starting with the wrong layout and waiting through cooldown can dominate the later policy choice. Remaining-work admission also renews whenever a closed-loop client begins another request. A present-cohort budget and a forecast of continuing demand are different quantities.

C. Forecasts and telemetry need careful denominators

Historical request disruptions grouped with executed decisions yield 34 estimated transitions. Compute-fault disruptions average 5,382 ms against a 3,054 ms forecast; network-fault disruptions average 3,948 ms. Measurements exceed forecast by 76.2% and 29.2%. Equivalently, the forecast is 43.3% and 22.6% below measurement. These percentages use different denominators and must not be interchanged. Historical grouping lacked explicit transition IDs; the 34 groups are not a directly logged transition population.

A separate synthetic loopback diagnostic finds 148.80 ms worker wait in a 149.98 ms call-minus-compute residual: 99.2%. It is synthetic evidence, not an Azure or home-network measurement. The real-model revision now records queue, compute and residual durations for each stage, allowing the same diagnosis on actual model execution. Subtracting queue time changes the telemetry estimate; it does not by itself prove a better placement or end-to-end outcome.

D. Resource-matched local comparisons

The new resource-matched matrix is incomplete. Its partial observations are retained in the artifact for continuation, but do not establish a complete repeated comparison. The optimized-engine and three-worker follow-ups are prepared but have not been executed.

E. Replay correctness and stalls

All 18 position-controlled recovery cases match the unsplit reference token sequence, including both active requests where concurrency is two. This checks 27 request outputs with 16 generated tokens each, not arbitrary fault schedules. The final sequences contain no duplicated or omitted tokens relative to that reference. Internally accepted token times show worst gaps of roughly 4–10 s across the tested cases. Exact output is compatible with a substantial user-visible pause.

Relayout waits to reload changed shards; loss reloads the full model on the survivor; restart rebuilds cache on an unchanged layer layout. Fault orchestration is manual and explicitly logged. The reported internal disruption interval begins when the router detects failure; maximum token gaps also include the orchestration delay. This experiment makes no claim about automatic detection latency or clients receiving streamed tokens.

VI. DISCUSSION AND LIMITS

For a model fitting on every worker, replicas and a justified unsplit CPU configuration should be measured before adopting a pipeline. The relevant baseline depends on resource budget, precision, dispatch, batching and model memory. A weak dispatcher can make pipeline overlap look better than it is. Our replicas use available endpoints, but do not implement a production batching engine. No state-of-the-art ranking follows.

Whole-run time is the primary endpoint for short sessions. Reporting only a fault phase can hide migration, warmup and drain costs. Cost models must account for every affected history and distinguish serial replay work from elapsed interruption. A live-context sum and explicit generation-linked records improve observability; they do not remove underforecasting or prove an uncertainty-aware controller is better. Same-generation cache recovery has request-scoped records, because its global transition identity is not known.

Concurrency changes the useful placement objective, but $Q > 1$ is insufficient to justify saturation for an arbitrary pipeline. The new envelope planner fixes an optimization error within the restricted model. Real-model evidence for three or more physical workers, natural heterogeneity and controller behavior at changing concurrency remains necessary.

The historical CPU model is small, prompts and output budgets are short, machines are similar, and faults are injected. The added context and concurrency matrix broadens local coverage while preserving these external-validity limitations. Ten restart blocks characterize some variation; they are not a universal power guarantee, and temporal or host interference can remain. Recovery depends on model-fit and excludes coordinator loss. The application demo is evidence of execution and control behavior, with its simulation mode clearly labeled; it is not a comparative performance experiment.

The strongest next studies are matched-start finite cohorts versus continuing arrivals, real transport measurements that change placement decisions, a held-out transition calibration study, an optimized external serving baseline, additional models, and genuinely different machines. Negative baseline results

should remain in the paper. More repetitions cannot establish algorithmic novelty or replace missing deployment conditions.

VII. CONCLUSION

The audited two-VM record shows that objective choice, transition accounting and failure recovery matter, while the payback gate has no demonstrated advantage over hysteresis. The revised runtime addresses a finite-concurrency planner error and exposes active replay histories, queue time and token stalls. New local baseline and correctness experiments make the small-model claim more testable. This is an empirical study of conditions and pitfalls, with broader deployment and external-system comparisons still required.

VIII. ARTIFACT AVAILABILITY

The writing bundle contains editable IEEE LaTeX and Markdown, figures, bibliography, experiment reports, source hashes and build instructions. The repository preserves historical raw matrices and the new local measurements separately. A stable public artifact URL and final author details must be provided according to the chosen venue’s policy. No submission or public release is implied by the local bundle.

REFERENCES

- [1] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. X. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, “GPipe: Efficient training of giant neural networks using pipeline parallelism,” in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 103–112.
- [2] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, P. B. Gibbons, and M. Zaharia, “PipeDream: Generalized pipeline parallelism for DNN training,” in *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, 2019, pp. 1–15.
- [3] S. H. Bokhari, “Partitioning problems in parallel, pipelined, and distributed computing,” *IEEE Transactions on Computers*, vol. 37, no. 1, pp. 48–57, 1988.
- [4] M. Zhang, X. Shen, J. Cao, Z. Cui, and S. Jiang, “Edgeshard: Efficient llm inference via collaborative edge computing,” *IEEE Internet of Things Journal*, vol. 12, no. 10, pp. 13 119–13 131, 2025. [Online]. Available: <https://doi.org/10.1109/JIOT.2024.3524255>
- [5] A. Borzunov, M. Ryabinin, A. Chumachenko, D. Baranchuk, T. Dettmers, Y. Belkada, P. Samygin, and C. Raffel, “Distributed inference and fine-tuning of large language models over the internet,” 2023. [Online]. Available: <https://arxiv.org/abs/2312.08361>
- [6] T. Sun, P. Wang, and F. Lai, “DiSCo: Device-server collaborative LLM-based text streaming services,” in *Findings of the Association for Computational Linguistics: ACL 2025*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 14 259–14 277. [Online]. Available: <https://aclanthology.org/2025.findings-acl.734/>
- [7] Z. Wang, Z. Hong, X. Li, Z. Wang, S. Li, Q. Meng, Q. Wang, C. Huan, R. Gu, S. Zhong, and C. Tian, “Star: Decode-phase rescheduling for llm inference,” in *Proceedings of the 35th International Symposium on High-Performance Parallel and Distributed Computing*, 2026, verified against final HPDC metadata printed in arXiv v2 PDF, page1; title changed from ARES in v1. [Online]. Available: <https://arxiv.org/abs/2510.13668>
- [8] Y. Fu, L. Xue, Y. Huang, A.-O. Brabete, D. Ustiugov, Y. Patel, and L. Mai, “ServerlessLLM: Low-latency serverless inference for large language models,” 2024, oSDI 2024 paper; cited arXiv version. [Online]. Available: <https://arxiv.org/abs/2401.14351>
- [9] X. Miao, C. Shi, J. Duan, X. Xi, D. Lin, B. Cui, and Z. Jia, “SpotServe: Serving generative large language models on preemptible instances,” 2023, aSPLOS 2024 paper; cited arXiv version. [Online]. Available: <https://arxiv.org/abs/2311.15566>
- [10] Y. Lin, B. Chen, X. Zhang, C. Xu, and K. Ye, “Dynopipe: Heterogeneous edge-cloud llm serving with dynamically orchestrated pipeline boundaries,” in *2026 ACM/IEEE 53rd Annual International Symposium on Computer Architecture (ISCA)*, 2026, pp. 969–984, metadata verified Crossref; primary full text inspected. [Online]. Available: <https://doi.org/10.1109/isca66397.2026.00077>
- [11] Y. Lin, S. Peng, C. Lu, C. Xu, and K. Ye, “Flexpipe: Adapting dynamic llm serving through inflight pipeline refactoring in fragmented serverless clusters,” in *Proceedings of the European Conference on Computer Systems (EuroSys)*, 2026, primary author manuscript, version 2, inspected 7 October 2026. [Online]. Available: <https://arxiv.org/html/2510.11938v2>
- [12] T. Berenbaum and M. Venkatachalam, “Pre-compiled pipeline shards for distributed LLM inference on intel AI PC fleets,” arXiv:2608.19147v1, 2026, preprint. [Online]. Available: <https://arxiv.org/abs/2608.19147>
- [13] R. Jain, *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. Wiley, 1991.
- [14] T. Hoeffler and R. Belli, “Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015, pp. 73:1–73:12.
- [15] J. D. C. Little, “A proof for the queuing formula: $L = \lambda W$,” *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.
- [16] M. Reiser and S. S. Lavenberg, “Mean-value analysis of closed multichain queuing networks,” *Journal of the ACM*, vol. 27, no. 2, pp. 313–322, 1980.